Software Engineering - Lab 3:
Design Patterns & Automated Testing

Objective: This assignment will challenge you to apply fundamental software design patterns to structure a robust application and to implement a comprehensive, multi-layered automated testing suite. You will write and automate unit, integration, and end-to-end (E2E) system tests within your CI/CD pipeline.

Context: We are building a new cross-platform To-Do List desktop application. The application's architecture is intentionally designed using the Model-View-Controller (MVC), Singleton, and Observer patterns to promote separation of concerns and testability. Your task is to implement a full suite of tests that validate the application's quality at every level and integrate these tests into your CI/CD pipeline.

Prerequisites

1. **Completed Assignments 1 & 2:** You should be comfortable with Git, GitHub, and the basic structure of a GitHub Actions workflow.
2. **Node.js and npm:** Must be installed.

Submission

1. The URL to your public GitHub repository containing the To-Do application.
   `https://github.com/<username>/git- todo-app-cicd-<StudentID>-<YourName>`

2. One ZIP named: `submission_lab3_<StudentID>_<YourName>.zip`
   with contents:

   1. A screenshot of a successful workflow run from the "Actions" tab showing the unit-and-integration-tests, and build jobs all passing.
      `ci_success_build_test_<StudentID>_<YourName>.png`

   2. Drawing of UML class diagram of the Model, View, and Controller class and submit either docx, pdf or image files.
      `uml_class_diagram_<StudentID>_<YourName>.pdf/docx/jpg/png`

Part 1: Project Setup & Understanding the Architecture

We will start with a new project structure that is pre-configured with a test-friendly architecture.

1. **Use a New Repository:** For this assignment, either create a new, clean repository on GitHub or create a new main branch in your existing repository to house this new project. With the name: `git-todo-app-cicd-<STUDENT-ID>-<FULLNAME>`

2. **Add Project Files:** Download and add the new todo-app-cicd project files to your local repository.
3. **Install Dependencies:** This project has new dependencies, including the Playwright framework for E2E testing. Run the install command:
   `npm install`

4. **Explore the Design Patterns:**
    - **Model-View-Controller (MVC):** Look at the js/ folder.
        - model.js: The **Model**. It handles the application's data and business logic. It knows nothing about the UI.
        - view.js: The **View**. It is responsible for rendering the UI and displaying data. It should not contain any business logic.
        - controller.js: The **Controller**. It acts as the intermediary, receiving user input from the View and calling the appropriate methods in the Model.
        - When you finished, create drawing of UML class diagram of the Model, View, and Controller class.
    - **Singleton:** Open js/model.js. The TodoService class is implemented as a Singleton. This ensures that only one instance of the service ever exists, providing a single, consistent source of truth for our to-do list data throughout the entire application.
    - **Observer:** The TodoService (the Subject) maintains a list of observers (in our case, just the View). When the to-do list data changes (e.g., an item is added), it calls the notify method, which in turn calls the update method on the View (the Observer), telling it to re-render the list. This decouples the Model from the View.

5. **Start the project:** Start the newly created Electron project with NPM. Run the install command:
```
npm run start
```

6. **Commit the Initial Project:**
```
git add .
git commit -m "Feat: Initial setup for To-Do application with MVC structure"
git push origin main
```

Part 2: Writing and Automating Unit Tests

Unit tests focus on the smallest piece of functionality—in our case, the methods within the TodoService model.

1. **Examine the Test File:** Open tests/unit/model.test.js. We have provided the structure for the tests using the Jest testing framework.
2. **Implement the Unit Tests:** Your task is to write the code for the test cases described. You need to test the addTodo, toggleTodoComplete, and removeTodo methods of the TodoService.
    - **Tip:** You will need to import the TodoService and create an instance of it. For each test, perform an action and then use Jest's expect() function to assert that the state of the todo array is what you expect it to be.
3. **Run Tests Locally:**
```
npm run test:unit
```
Ensure all tests pass.
4. **Commit Your Tests:**
```
git add .
git commit -m "Test: Implement unit tests for TodoService"
git push
```

Part 3: Writing and Automating Integration Tests

Integration tests check how multiple units work together. We will test the interaction between the Controller and the Model.

1. **Examine the Test File:** Open tests/integration/controller.test.js.
2. **Implement the Integration Tests:** Complete the test cases. Here, you will be creating instances of the TodoService and the Controller. The goal is to call controller methods (which simulate user actions) and then assert that the data in the TodoService model has changed correctly.
3. **Run Tests Locally:** You can run Jest integration tests with:
   `npm run test:integration`

4. **Commit Your Tests:**

```
git add .
git commit -m "Test: Implement integration tests for Controller-Model interaction"
git push
```

Part 4: Writing System (End-to-End) Tests

End-to-end (E2E) tests validate the entire application workflow from the user's perspective. We will use Playwright to launch our Electron application and simulate user interactions.

1. **Examine the E2E Test File:** Open tests/e2e/app.spec.js. This file uses Playwright's syntax.
2. **Implement the E2E Test:** Complete the test scenario. The comments will guide you through the steps:
   - Launch the Electron application.
   - Locate the input field and type a new to-do item.
   - Click the "Add" button.
   - Verify that the new item appears in the list.
   - Find the checkbox for that item and click it.
   - Verify the item now has the 'completed' style.
   - Find and click the "Delete" button for that item.
   - Verify that the item has been removed from the list.
3. **Run E2E Tests Locally:**
   `npm run test:e2e`

   A window for the application will pop up and you will see the test script interacting with it automatically.
4. **Commit Your E2E Tests:**

```
git add .
git commit -m "Test: Implement E2E test for core user workflow"
git push
```

Part 5: Updating the CI/CD Pipeline

Now, integrate all your automated tests into the CI/CD pipeline to ensure that every push is automatically validated.

1. **Create the Workflow File:** Create a .github/workflows/main.yml file.
2. **Define the Comprehensive Workflow:** This workflow will run all three types of tests in separate jobs. The build job will only run if all tests pass.

Copy this content into main.yml:

```
name: Full Test and Build Pipeline

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  unit-and-integration-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - run: npm install
      - name: Run Jest Tests
        run: npm test


   build:
     needs: [unit-and-integration-tests] # Depends on all tests passing
     runs-on: ubuntu-latest
     steps:
       - uses: actions/checkout@v4
       - uses: actions/setup-node@v4
         with:
           node-version: '20'
       - run: npm install
       - name: Build application
         run: npm run build
       - name: Upload artifact
         uses: actions/upload-artifact@v4
         with:
           name: todo-app-linux
           path: dist/
```

3. **Commit and Push:** Save the workflow file, commit it, and push it to GitHub. Go to the "Actions" tab to watch your complete testing pipeline run.

```
git add .
git commit -m "Build: add workflow for artifact build"
git push
```

Grading Rubric

Part	Criteria	Points
1. Project Setup & Architecture	Repo created and pushed as `git-todo-app-cicd-<StudentID>-<FullName>` (public)	5
	UML class diagram of Model-View-Controller submitted (docx/pdf/image) `uml_class_diagram_<StudentID>_<YourName>.pdf/docx/jpg/png`	15
	Subtotal	**20**
2. Unit Testing (Jest)	Unit tests for 4 tests implemented (2 points each)	8
	All tests passed (2 points each)	8
	Proper assertions and structure following Jest conventions	4
	Subtotal	**20**
3. Integration Testing (Controller–Model)	Integration tests for 2 tests implemented (3 points each)	6
	All tests passed (3 points each)	6
	Proper assertions and structure following Jest conventions	3
	Subtotal	**15**
4. End-to-End (E2E) Testing (Playwright)	E2E tests for 4 parts implemented (3 points each)	12
	Test passed (8 points each)	8
	Proper Playwright assertions and structure	5
	Subtotal	**25**
5. CI/CD Workflow Integration	Workflow file at `.github/workflows/main.yml` exists and runs	10
	Screenshot of successful GitHub Actions run showing all jobs passing `ci_success_build_test_<StudentID>_<YourName>.png`	10
	Subtotal	**20**
	TOTAL	**100**

Good luck!